

Projekt: Krasnal

Eryk Daczyński
Małgorzata Maćkowiak
Jeremiasz Waclawski
Aleks Zieliński

May 11, 2026

Contents

1	Informacje wstępne	3
1.1	Treść projektu	3
1.2	temp	4
2	Funkcje	4
2.1	det()	4
2.2	cmpDouble() i sgn()	5
2.3	is_point_on_segment()	5
2.4	do_segments_intersect()	5
2.5	distance()	6
2.6	is_point_in_polygon()	6
2.7	graham_scan()	7
3	Struktury	8
3.1	Point	8
4	Testy	9
5	Prezentacja działania	9

1 Informacje wstępne

1.1 Treść projektu

Pracując nad poniższym projektem, należy: odfiltrować informacje istotne od nieistotnych sformalizować problem i podać jego specyfikację; do rozwiązania wykorzystać odpowiednie struktury danych oraz algorytmy; opisać teoretycznie rozwiązanie; przeanalizować jego optymalność oraz złożoność czasową/pamięciową; zaimplementować rozwiązanie w dowolnym języku programowania; przeprowadzić testy na różnych przykładach.

Problem

Odkąd królowa Śnieżka wraz z księciem z bajki rządzą krasnoludkami, ich królestwo nabiera siły i rośnie. Obecnie zamieszkuje je o wiele więcej niż tylko siedmiu krasnoludków. Każdy z nich jest specjalistą w swoim fachu i w kopalniach lub wyrobiskach potrafi wydobywać różne minerały. Królowa zna każdego ze swoich poddanych i wie jakie minerały każdy z nich potrafi wydobywać. To ona decyduje o miejscu pracy każdego z krasnoludków, jest to dla niej niełatwym zadaniem. Chcąc dbać o swoich obywateli, zapewniając im dostatek, królowa zleciła badania geologiczne w swoim kraju. Dzięki temu ma informacje o położeniu poszczególnych złóż w swoim kraju i wie, jaka jest ich wydajność (ilu krasnoludków może pracować w tym miejscu). Okazuje się, że najważniejsza jest miłość do pracy, którą się wykonuje. To znaczy, że jeśli krasnoludek pracuje przy materii zgodnej z jego preferencjami, wydobydzie kruszec o stałej wartości, niezależnie czy kopie złoto, węgiel, rudę miedzi, czy inny surowiec. Królowa Śnieżka poprosiła, abyś na podstawie posiadanych przez nią informacji pomógł jej przydzielić pracę każdemu z krasnoludków.

Po rozmowie z księciem okazało się, że jego ukochana bardzo dużo czasu spędza w kuchni, nadzorując gotowanie owsianki dla wygłodniałych brodaczy. Ilość potrzebnej owsianki zwiększa się, gdy krasnoludki muszą pokonać większą odległość z domu do pracy (można przyjąć, że jest to jedyny czynnik). Książę, chcąc ulżyć swojej wybrance, zmierzył i spisał odległości, pomiędzy każdym z wyrobisk i domkiem każdego z krasnoludków. Prosi, abyś uwzględnił te informacje. Przy podziale pracy chciałby, aby sumaryczna odległość, którą każdego dnia muszą pokonać krasnoludki była możliwie jak najmniejsza, przy czym nie możemy pozwolić, żeby spadła wartość produkowanych dóbr.

Niestety nie jest to jedyny problem. Królowa Śnieżka po incydencie ze złą królową jest nieco drażliwa, gdy pojawia się temat jabłek. Nakazała wyciąć wszystkie jabłonie w królestwie i zabroniła spożywania owoców. Krnąbrne krasnoludki starają się jednak wzbogacić swoją dietę o pyszne renety, antonówki, cortlandy, papierówki i ligole. Szmuglują przez granice królestwa pewne ilości tych owoców. W celu egzekucji prawa, zakazującego spożywania jabłek na terenie królestwa, książę codziennie rano musi okrążyć teren, na którym znajdują się wszystkie użytkowane kopalnie. Chciałby, żeby ta codziennie pokonywana odległość była możliwie mała. Gdy i to nie przynosiło skutku, książę wzdłuż trasy swego przejazdu rozstawił dekametrowców (krasnoludki uzbrojone w łuki ustawione co określoną liczbę metrów). Mają one zapobiec przerzucaniu jabłek przez granicę. Ataki takie wciąż zdarzają się często, atakowany jest dany wielometrowy odcinek granicy, przez który leci niezliczona liczba jabłek w jednym momencie. Dekametrowcy nie mogą opuścić swojego odcinka granicy, a najskuteczniejszą obroną przed lecącymi jabłkami okazuje się salwa.

Najgłośniejszy z krasnoludków atakowanej części granicy powinien wydać sekwencję poleceń:

„strzały na cięciwy
- naciągnąć cięciwy
- strzał!”.

Głos krasnoludka jest jego dumą i sprawą drażliwą, niełatwo więc przychodzi im ustalenie najgłośniejszego. Zaproponuj szybki sposób znajdowania dekametrowca z danego kawałka granicy, który winien wydać rozkazy.

Jak widać, królowa i książę muszą wykonać ogrom pracy, wykorzystując skomplikowane działania. Martwią się, że ta wiedza zaginie. Pomóż im zapisać cenną wiedzę w elektronicznych księgach, oszczędzając elektroniczny papier. Zatroszcz się, żeby potomni mogli szybko

wyszukiwać potrzebne informacje.

Co powinny przygotować zespoły studenckie

W ramach projektu należy przygotować rozwiązanie problemu przedstawionego powyżej.

Rozwiązanie powinno składać się z:

- dokumentacji
- plików programów
- opisu poprawności rozwiązania
- raportów przeprowadzonych testów

Wyniki pracy powinny być zapisywane z wykorzystaniem systemu Git (prowadzący powinien mieć dostęp do repozytorium). Projekt będzie oceniany zgodnie z informacjami przedstawionymi na USOS. Każda osoba będzie musiała wypełnić dwie ankiety: opisującą jej wkład w rozwiązanie problemu oraz oceniającą kompletność rozwiązania problemów przez zespół, do którego należy.

1.2 temp

```
1 int main(){
2     int i = 0;
3
4     return i;
5 }
```

Plik źródłowy 1: x example

2 Funkcje

2.1 det()

Funkcja pozwalająca ustalić położenie punktu p_3 względem wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) > 0$, to p_3 leży po lewej stronie wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) < 0$, to p_3 leży po prawej stronie wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) = 0$, to punkty p_1, p_2, p_3 są współliniowe

```
1 #include "det.h"
2
3 double det(Point p1, Point p2, Point p3) {
4     // 3 x 3
5     // D = (x2 - x1)(y3 - y1) - (y2 - y1)(x3 - x1)
6
7     /*
8         | 1 x1 y1 | = p1
9         | 1 x2 y2 | = p2
10        | 1 x3 y3 | = p3
11
12        D > 0 => p3 na lewo od p1p2
13        D < 0 => p3 na prawo od p1p2
14        D = 0 => p3 "na" p1p2 (współliniowy)
15
16     */
17
18     return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x);
19 }
```

Plik źródłowy 2: /Struktury/Funkcje/det.cpp

2.2 cmpDouble() i sgn()

cmpDouble(a, b) sprawdza która liczba jest większa z uwzględnieniem błędu typu liczbowego double
sgn(x) sprawdza znak liczby *x*, korzystając z *cmpDouble(x, 0.0)* [dokładniej, to sprawdza czy liczba jest większa od 0]

```
1 #ifndef EPS_H
2 #define EPS_H
3
4 constexpr double EPS = 1e-9;
5
6 inline int cmpDouble(double a, double b) {
7     if (a < b - EPS) return -1;
8     if (a > b + EPS) return 1;
9     return 0;
10 }
11
12 inline int sgn(double x) {
13     return cmpDouble(x, 0.0);
14 }
15
16 #endif
```

Plik źródłowy 3: /Struktury/Funkcje/EPS.h

2.3 is_point_on_segment()

W tej funkcji sprawdzamy przynależność punktu *P* do odcinka p_1p_2

Pierw sprawdzamy czy punkt *P* jest współliniowy z $\overrightarrow{p_1p_2}$, jeżeli nie to nie należy do odcinka p_1p_2

Następnie jeżeli jest współliniowy, to sprawdzamy kolejno współrzędne *x* i *y* czy spełniają warunki odpowiednio:

$$\min(p_1.x, p_2.x) \leq P.x \leq \max(p_1.x, p_2.x)$$

$$\min(p_1.y, p_2.y) \leq P.y \leq \max(p_1.y, p_2.y)$$

Jeżeli spełnione są oba te warunki to punkt *P* należy do odcinka p_1p_2

```
1 #include "is_point_on_segment.h"
2
3 // Sprawdzenie czy punkt P należy do odcinka p1p2
4 bool is_point_on_segment(Point p1, Point p2, Point P)
5 {
6     // P należy gdy p1 p2 i P sa wspolliniowe -> D = 0
7     if(sgn(det(p1, p2, P)) != 0){
8         return false;
9     }
10    // P należy gdy min(x1,x2) <= x3 <= max(x1,x2)
11    // oraz max(y1, y2) <= y3 <= max(y1, y2)
12    bool x_ok = cmpDouble(std::min(p1.x, p2.x), P.x) <= 0 &&
13                cmpDouble(P.x, std::max(p1.x, p2.x)) <= 0;
14    bool y_ok = cmpDouble(std::min(p1.y, p2.y), P.y) <= 0 &&
15                cmpDouble(P.y, std::max(p1.y, p2.y)) <= 0;
16
17    return x_ok && y_ok;
18 }
```

Plik źródłowy 4: /Struktury/Funkcje/is_point_on_segment.cpp

2.4 do_segments_intersect()

Funkcja sprawdza czy odcinki p_1p_2 i p_3p_4 przecinają się

Obliczamy $D_1 = \det(p_1, p_2, p_3)$, $D_2 = \det(p_1, p_2, p_4)$, $D_3 = \det(p_3, p_4, p_1)$ i $D_4 = \det(p_3, p_4, p_2)$

Jeżeli $D_1 \cdot D_2 < 0$ i $D_3 \cdot D_4 < 0$ to p_1p_2 i p_3p_4 przecinają się

Jeżeli $D_1 = 0$ lub $D_2 = 0$ lub $D_3 = 0$ lub $D_4 = 0$ to sprawdzamy przynależności punktu do prostej (ponieważ $\overrightarrow{p_1p_2}$ i $\overrightarrow{p_3p_4}$ są współliniowe)

```

1 #include "do_segments_intersect.h"
2
3 // Sprawdzenie czy odcinki p1p2 p3p4 sie przecinaja
4 bool do_segments_intersect(Point p1, Point p2, Point p3, Point p4)
5 {
6     double d1 = det(p1,p2,p3);
7     double d2 = det(p1,p2,p4);
8     double d3 = det(p3,p4,p1);
9     double d4 = det(p3,p4,p2);
10
11     // Przecinaja sie gdy:
12     // WARUNEK 2: kazdy odcinek przecina prosta wyznaczona przez drugi odcinek
13     if((sgn(d1) * sgn(d2) < 0) && (sgn(d3) * sgn(d4) < 0)) return true;
14
15     // WARUNEK 1: ktorys z koncow odc. nalezy do drugiego odc.
16     // Czy P1 lezy na P3P4 || Czy P2 lezy na P3P4 || Czy P3 lezy na P1P2 || Czy P4
17     // lezy na P1P2
18     if(is_point_on_segment(p3, p4, p1) || is_point_on_segment(p3, p4, p2) ||
19        is_point_on_segment(p1, p2, p3) || is_point_on_segment(p1, p2, p4)) {
20         return true;
21     }
22     return false;
23 }

```

Plik źródłowy 5: /Struktury/Funkcje/do_segments_intersect.cpp

2.5 distance()

Funkcja oblicza kwadrat odległości punktu a od punktu b według metryki euklidesowej

```

1 #include "distance.h"
2
3 double distance(Point a, Point b) {
4     return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
5 }

```

Plik źródłowy 6: /Struktury/Funkcje/distance.cpp

2.6 is_point_in_polygon()

Funkcja implementuje algorytm z wykładu sprawdzający czy punkt p należy do listy wierzchołków (punktów *Point*) wielokąta *polygon*

Ustalmy $u = \min(x : x \in \text{polygon})$, $v = \max(x : x \in \text{polygon})$

Jeżeli $p.x < u$ lub $p.x > v$ to p nie leży w wielokącie *polygon*

Jeżeli p leży na dowolnej krawędzi wielokąta *polygon* to p leży w wielokącie *polygon*

Niech $q = (v + 1, p.y)$ i $l = 0$, i dla każdej krawędzi $p.y$ należy do domknięto-otwartego przedziału współrzędnych y tej krawędzi oraz pq ją przecina, to zwiększ l o

Jeżeli l jest nieparzyste, to punkt p leży w wielokącie *polygon*

```

1 #include "is_point_in_polygon.h"
2
3 bool is_point_in_polygon(Point p, const std::vector<Point>& polygon){
4     double u = polygon[0].x;
5     double v = polygon[0].x;
6     int n = polygon.size();
7     for(int i = 0; i < n; i++){
8         u = std::min(u, polygon[i].x);
9         v = std::max(v, polygon[i].x);
10    }
11
12    if((p.x < u) || (p.x > v)) return false;
13
14    int l = 0;
15

```

```

16 Point q = Point(v + 1, p.y);
17
18 for(int i = 0; i < n-1; i++){
19     if(is_point_on_segment(polygon[i], polygon[i+1], p)) return true;
20     if((p.y >= std::min(polygon[i].y, polygon[i+1].y)) && (p.y < std::max(polygon
21 [i].y, polygon[i+1].y))){
22         if(do_segments_intersect(polygon[i], polygon[i+1], p, q)) l++;
23     }
24 }
25
26 if(l % 2 == 1) return true;
27 return false;
28 }

```

Plik źródłowy 7: /Struktury/Funkcje/is_point_in_polygon.cpp

2.7 graham_scan()

```

1 #include "graham_scan.h"
2
3 // https://en.wikipedia.org/wiki/Graham_scan
4 std::vector<Point> graham_scan(std::vector<Point> points){
5     /// KROK 1: USUNIECIE DUPLIKATOW I USTAWIENIE NAJBЛИЗSZEGO
6     /// PUNKTU JAKO PIERWSZY (NAJPIERW NA Y, W RAZIE REMISU NA X)
7
8     std::sort(points.begin(), points.end(), [](const Point& a, const Point& b) {
9         int cmpY = cmpDouble(a.y, b.y);
10        if (cmpY != 0) return cmpY < 0;
11        return cmpDouble(a.x, b.x) < 0;
12    });
13
14    points.erase(std::unique(points.begin(), points.end(), [](const Point& a, const
15 Point& b) {
16        return cmpDouble(a.x, b.x) == 0 && cmpDouble(a.y, b.y) == 0;
17    }), points.end());
18
19    /// PRZYPADKI, GDY JEST PONIZEJ 3 PUNKTOW
20    if (points.size() < 3) {
21        if (points.size() == 0) return {};
22        if (points.size() == 1 || points.size() == 2) return points;
23    }
24
25    /// KROK 2: SORTUJEMY I DOSTOSOWUJEMY LISTE PUNKTOW ZE WZGLEDU NA WSPOLRZEDNE
26    KATOWE
27
28    /// NAJPIERW SORTUJEMY LISTE PUNKTOW
29    std::sort(points.begin() + 1, points.end(), [&](const Point& pi, const Point& pj)
30    {
31        double d = det(points[0], pi, pj);
32        if (sgn(d) == 0) {
33            int cmpDist = cmpDouble(distance(points[0], pi), distance(points[0], pj))
34            ;
35            if (cmpDist != 0) return cmpDist < 0;
36            int cmpX = cmpDouble(pi.x, pj.x);
37            if (cmpX != 0) return cmpX < 0;
38            return cmpDouble(pi.y, pj.y) < 0;
39        }
40        return sgn(d) > 0;
41    });
42
43    /// USUWAMY Z LISTY BLIZSZE PUNKTY Z TA SAMA WSPOLRZEDNA KATOWA CO DALSZY PUNKT
44    std::vector<Point> furtherPoints;
45    furtherPoints.push_back(points[0]);
46
47    for(unsigned int i = 1; i < points.size(); i++){
48        while(i + 1 < points.size() && sgn(det(points[0], points[i], points[i+1])) ==
49 0)

```

```

45         i++;
46         furtherPoints.push_back(points[i]);
47     }
48
49     points = furtherPoints;
50
51     /// KROK 3: Utworzenie stosu i dodanie 3 pierwszych punktów
52
53     if (points.size() < 3) return points;
54
55     std::vector<Point> stack;
56     stack.push_back(points[0]);
57     stack.push_back(points[1]);
58     stack.push_back(points[2]);
59
60     /// KROK 4: Ostateczne dobranie punktów otoczki
61
62     for (unsigned int i = 3; i < points.size(); i++) {
63         while (stack.size() >= 2 && sgn(det(stack[stack.size() - 2], stack.back(),
64         points[i])) <= 0) {
65             stack.pop_back();
66         }
67         stack.push_back(points[i]);
68     }
69     return stack;
70 }

```

Plik źródłowy 8: /Struktury/Funkcje/graham_scan.cpp

3 Struktury

3.1 Point

```

1 #include "Point.h"
2
3 Point::Point(double x, double y) : x(x), y(y) {}
4
5 // Operator mniejszosci
6 bool Point::operator<(const Point& other) const {
7     int cmpX = cmpDouble(x, other.x);
8     if (cmpX != 0) {
9         return cmpX < 0;
10    }
11    return cmpDouble(y, other.y) < 0;
12 }
13
14 // Operator wiekszosci
15 bool Point::operator>(const Point& other) const {
16     return other < *this;
17 }
18
19 // Operator rownosci
20 bool Point::operator==(const Point& other) const {
21     return cmpDouble(x, other.x) == 0 && cmpDouble(y, other.y) == 0;
22 }
23
24 // Operator nierownosci
25 bool Point::operator!=(const Point& other) const {
26     return !(*this == other);
27 }
28
29 // Operator mniejszosci lub rownosci
30 bool Point::operator<=(const Point& other) const {
31     return (*this < other) || (*this == other);
32 }
33

```



```

34 // Operator większości lub równości
35 bool Point::operator>=(const Point& other) const {
36     return (*this > other) || (*this == other);
37 }
38
39 // Operator przypisania
40 Point& Point::operator=(const Point& other) {
41     if(this != &other){
42         x = other.x;
43         y = other.y;
44     }
45
46     return *this;
47 }
48
49 // Wypisanie punktu
50 std::ostream& operator<<(std::ostream& stream, const Point& p){
51     stream << "(" << p.x << ", " << p.y << ")";
52     return stream;
53 }

```

Plik źródłowy 9: Point.cpp

4 Testy

5 Prezentacja działania